MLDS 490 Advanced Algorithms for Artificial Intelligence

Assignment #1

Yuxuan Liang
GitHub username: `yuxuanliang1`
Repository name: `MLDS490-Assignment1`
Repository URL: `https://github.com/yuxuanliang1/MLDS490-Assignment1`

Abstract

This report studies exact dynamic programming for a discounted shuttle-dispatch MDP and Monte Carlo policy gradients for CartPole and Pong. Independently validated value and policy iteration agree on the shuttle policy: wait at states 0–12 and dispatch at 13–200. REINFORCE learns a strong primary CartPole policy; adding a running constant baseline raises its 500-episode evaluation mean from 429.248 to 497.282. Training-seed variability remains substantial. Both 1,000-game Pong experiments remain close to losing every game: their 500-game means are −20.608 and −20.850. Paired fixed-policy diagnostics show lower gradient covariance trace with the baseline in both games, demonstrating that variance reduction and final performance are distinct outcomes. Mathematical checks, independent numerical references, checkpoint restoration and explicit estimator conventions support the implementation; the evidence does not establish reliable Pong learning.

Contents

Experimental scope and reading guide **3**

1 Value and policy iteration **3**

1.1 MDP formulation and modeling assumptions . . . 3
1.2 Algorithms and numerical safeguards . . . 4
1.3 Main numerical findings . . . 4
1.4 Correctness experiments A–D: independent numerical evidence . . . 7
1.5 Experiments E–G: demand, costs, and discounting . . . 8
1.6 Experiments H–I: simulation and action gaps . . . 9
1.7 Edge cases and numerical sensitivity . . . 11
1.8 Reproduction, interpretation, and limitations . . . 11
1.9 Validation plan for an additional three months . . . 12

2 Policy Gradient: Part 1 **13**

2.1 Estimator, implementation, and objective convention . . . 13
2.2 CartPole-v1 . . . 13

2.3 Pong-v5: environment, inputs, and pilots . . . 16
2.4 Quantitative correctness and edge cases . . . 18
2.5 Limitations and an additional three months . . . 19

3 Policy Gradient: Part 2 — running constant baseline 19
3.1 Method and matched experimental design . . . 19
3.2 Why a constant baseline preserves the expected estimator . . . 20
3.3 CartPole training and independent evaluation . . . 20
3.4 Completed Pong training and independent evaluation . . . 23
3.5 Quantitative correctness and gradient variance . . . 25
3.6 Edge cases and reproducibility . . . 25
3.7 Interpretation, limitations and three-month validation plan . . . 27

4 Cross-experiment interpretation and extended validation 28
4.1 What the results establish . . . 28
4.2 Production-path audit of Pong . . . 28
4.3 A coordinated hypothetical three-month plan . . . 30

A Code Execution and Reproducibility 30

B Supporting dynamic-programming figures 32

Experimental scope and reading guide

The assignment combines a finite-state discounted-cost problem with two sample-based control tasks. Dynamic programming uses a known transition model; the policy-gradient experiments learn directly from environment interaction. The Bellman and score-function formulations follow standard reinforcement learning foundations [1, 2]. All numerical results in this report come from completed saved experiments. The three-month studies described later are proposals, not additional completed experiments.

Environment	γ	Adam rate	Episodes	Batch	Weight convention
CartPole-v1	0.95	0.003	1,500	10	G_t or $G_t - b$
ALE/Pong-v5	0.99	0.01	1,000	5	$\gamma^t G_t$ or $\gamma^t (G_t - b)$

Table 1: Matched primary training settings in Parts 1 and 2. Part 2 adds only the running constant baseline, with EMA coefficient 0.05. CartPole's rate was selected using Part 1 training-only pilots; Part 2 reuses that selection.

The primary policy-gradient training seed is 11. Each final checkpoint is evaluated in a separate process for exactly 500 stochastic episodes, with episode reset seeds $1000000 + i$, $i = 0, \ldots, 499$. No final evaluation is used to select a checkpoint. CartPole episodes end at failure or the normal 500-step limit; Pong evaluations contain 500 terminated games and no truncations. Reported SDs are sample standard deviations with denominator $n - 1$. Evaluation uncertainty is conditional on a trained policy; it does not quantify variability across training seeds. Separate CartPole diagnostics use 100 episodes per seed and are never pooled into the primary evaluation.

Table 1 distinguishes the estimator conventions; the full evaluation comparison is in Table 10. Section 1 provides the MDP and independent optimality evidence; Sections 2 and 3 describe the two policy-gradient methods. Section 4 interprets the Pong findings and the limits of the evidence.

The required shuttle plots (Figures 1, 2 and 3) show close finite/infinite-horizon values and a switching policy at state 13. Figures 4 and 5 support the quantitative horizon and policy-separation arguments; numerical validation, rather than visual agreement alone, underpins these conclusions.

1 Value and policy iteration

1.1 MDP formulation and modeling assumptions

The system is a finite, discounted-cost Markov decision process (MDP). At each decision epoch, $s \in \{0, \ldots, 200\}$ passengers are waiting and the controller chooses $a \in \{0, 1\}$, where 0 means wait and 1 dispatches one shuttle. For capacity $K = 15$, fixed dispatch cost $c_f = 100$, holding cost $c_h = 2$, and discount factor $\gamma = 0.95$, define

$$r(s, a) = \max(0, s - Ka), \qquad c(s, a) = c_f a + c_h r(s, a), \tag{1}$$

$$s' = \min(200, r(s, a) + A), \qquad \Pr(A = k) = 1/5, \quad k = 1, \ldots, 5. \tag{2}$$

For repeated capped successors, their arrival probabilities are added when constructing $P_a(s, s')$. The objective is

$$V^*(s) = \min_\pi \mathbb{E}_s^\pi \left[\sum_{t=0}^{\infty} \gamma^t c(s_t, a_t) \right].$$

All costs below are in abstract cost units and time is measured in decision periods.

The event order is action, holding-cost assessment, then arrivals. Thus newly arrived passengers do not pay holding cost until the next decision. A dispatch carries up to K passengers; a partial load and an empty dispatch are both mathematically permitted. Empty dispatch still costs c_f and does not change the successor distribution. Only one shuttle can depart per epoch. The state cap truncates arrivals without a rejection penalty: excess passengers disappear from this model. This assumption changes incentives near the boundary and is not a physical conservation law. The finite-horizon terminal value is zero, so passengers left after the final decision have no later cost. These timing, overflow, and terminal conventions are explicit modeling choices; the results are conditional on them.

1.2 Algorithms and numerical safeguards

Finite-horizon enumeration. Exact backward induction enumerates the two actions at each state and epoch, rather than enumerating 2^{500} action sequences:

$$V_T(s) = 0, \qquad V_t(s) = \min_{a\in\{0,1\}} \left[c(s,a) + \gamma \sum_{s'} P_a(s,s') V_{t+1}(s') \right].$$

The program saves all 501 value rows and all 500 policy rows for $T = 500$, including V_0 and the time-zero policy. Arrival-support backups cost $O(T|\mathcal{S}||\mathcal{A}||\operatorname{supp}(A)|)$; storing the complete finite-horizon solution costs $O(T|\mathcal{S}|)$.

Value iteration (VI). Starting from $V_0 = 0$, VI repeatedly applies the Bellman minimum operator $\mathcal{B}$. It stops when $\|V_{k+1} - V_k\|_\infty \le 10^{-10}$ and then computes a fresh Bellman residual. Since $\gamma < 1$, $\mathcal{B}$ is a sup-norm contraction. Nonnegative costs make zero a lower bound, so exact iterates increase to V^*. Residuals certify value error through

$$\|V - V^*\|_\infty \le \frac{\|V - \mathcal{B}V\|_\infty}{1-\gamma}.$$

Convergence histories and runtimes are saved, including the logarithmic convergence figure in the figure directory.

Howard policy iteration (PI). PI starts with the never-dispatch policy, evaluates it with a numerical linear system solver,

$$(I - \gamma P_\pi) V_\pi = c_\pi,$$

and improves all actions using $c(s,a) + \gamma P_a V_\pi$. The reported pass count includes the final evaluation/improvement pass that confirms stability; thus six passes need not mean six policy changes. Termination requires both a stable policy and Bellman residual at most 10^{-9}.

Both algorithms prefer wait when action values differ by at most eight machine epsilons times their local scale. This band handles floating-point ties rather than imposing a large absolute tolerance. PI checks the unmodified Bellman minimum; if stability fails its residual certificate, it retries strict minimization and raises an error if evaluation precision is insufficient. An explicit 10^{-9}-advantage perturbation test confirms that a small, resolvable improvement is taken.

1.3 Main numerical findings

VI required 519 backups and PI 6 evaluation/improvement passes (Table 2). They selected the same action at every state. Their maximum and mean absolute value differences were 1.865×10^{-9} and 1.864×10^{-9},

respectively. The PI values were $V^*(0) = 576.670168$ and $V^*(200) = 3821.586477$. Values shown as optimal are numerical approximations with the reported residual certificates.

Algorithm	Backups/passes	Residual	Runtime (s)
Value iteration	519	9.413×10^{-11}	0.0097
Policy iteration	6	9.095×10^{-13}	0.0177

Table 2: Executed baseline solver results. Runtimes are illustrative and depend on hardware and the numerical linear algebra implementation.

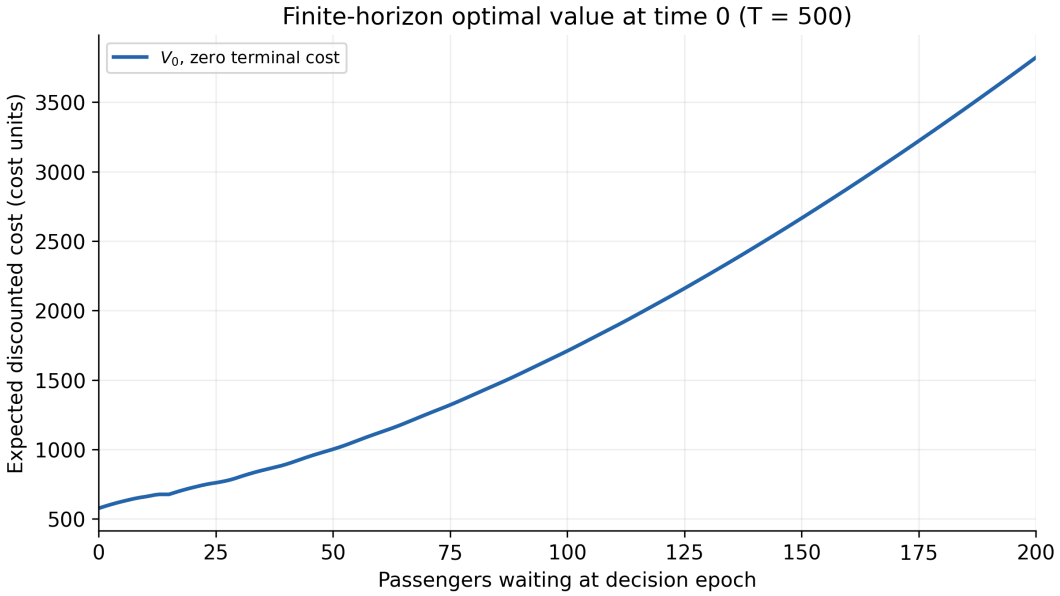


Figure 1: Required plot (a): time-zero optimal cost for the 500-period problem, with zero terminal value. The stored policies may depend on the epoch.

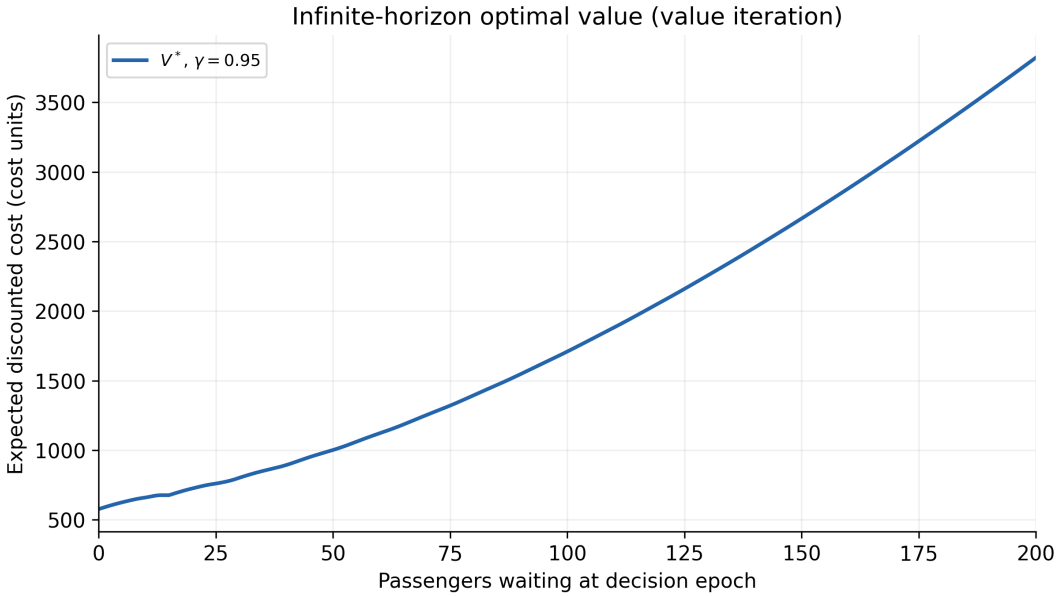


Figure 2: Required plot (b): infinite-horizon optimal cost computed by VI. Its visual agreement with Figure 1 is supported quantitatively by the horizon error experiment.

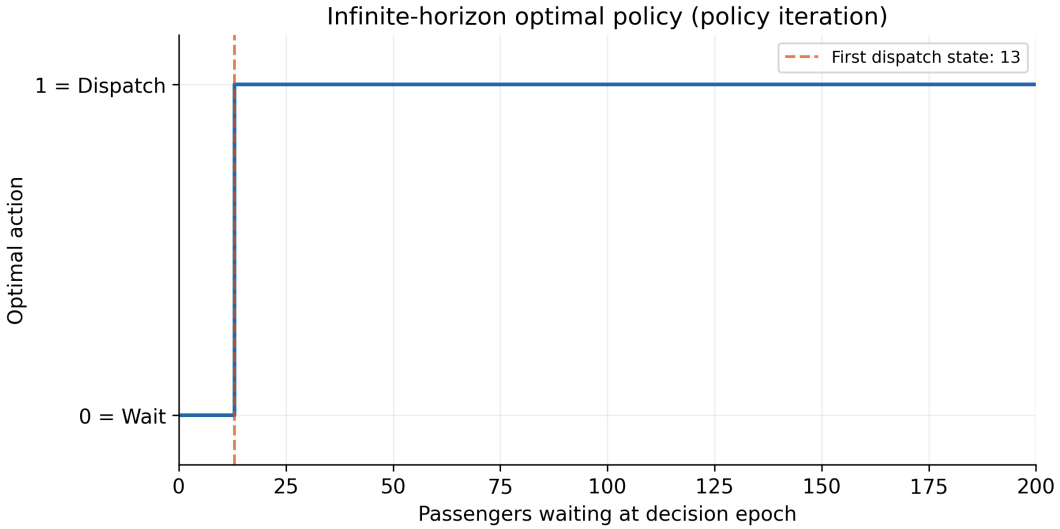


Figure 3: Required plot (c): PI chooses wait at states 0–12 and dispatch at states 13–200. A single switching state was detected, rather than imposed.

1.4 Correctness experiments A–D: independent numerical evidence

A: VI versus PI. The two methods solve the infinite-horizon problem using different updates. There were zero policy disagreements. The minimum absolute action-value gap was 3.778 (at state 12), much larger than the value-error certificates; agreement is therefore not an artifact of nearly tied baseline actions.

B: finite versus infinite horizon. For zero terminal cost and nonnegative bounded costs,

$$0 \le V^*(s) - V^{(T)}(s), \qquad \|V^* - V^{(T)}\|_\infty \le \gamma^T \frac{C_{\max}}{1-\gamma}.$$

Here $V^{(T)}$ denotes time-zero value for horizon T, not the terminal row V_T. The valid all-action cost bound is $C_{\max} = \max\{2(200), 100 + 2(200 - 15)\} = 470$. All six observed errors decrease and satisfy the bound (Table 3). At $T = 500$, the error is 4.940×10^{-9} and the bound 6.838×10^{-8}: $0.95^{500} \approx 7.27 \times 10^{-12}$ explains the close approximation. Comparisons use the PI reference and allow its residual error plus a conservative floating-point accumulation allowance, both saved in the CSV. The observed errors also satisfy the uninflated tail bounds.

T	Observed sup error	Discounted-tail bound
5	2011.834	7273.541
10	917.018	5628.127
25	188.417	2607.462
50	52.249	723.283
100	4.020	55.653
500	4.940×10^{-9}	6.838×10^{-8}

Table 3: Experiment B: errors against PI and valid theoretical tail bounds.

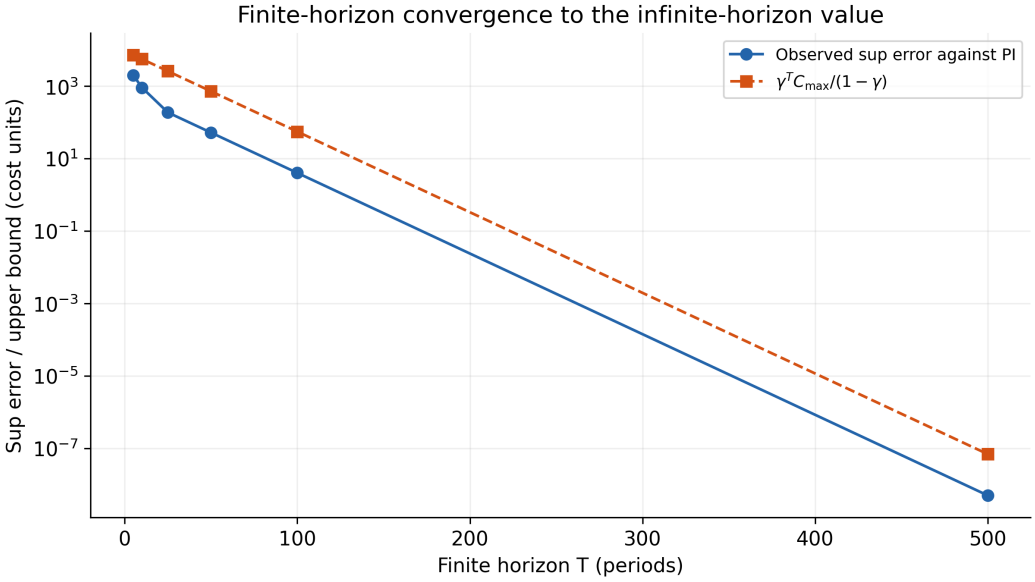


Figure 4: Observed finite-horizon sup error and theoretical upper bound on a logarithmic vertical scale. Both are expressed in cost units.

C: independent Bellman residuals. A scalar reference implementation reconstructs transitions and costs directly from the primitive model parameters, without the production backup arrays. It gives $R(V_{\mathrm{VI}}) = 9.413 \times 10^{-11}$ and $R(V_{\mathrm{PI}}) = 9.095 \times 10^{-13}$. The fixed-policy equation residual, evaluated on the directly solved VI policy using independently rebuilt transitions, is 9.095×10^{-13}. These checks test both optimality and consistency of policy evaluation.

D: exhaustive stationary policies on a small MDP. With $S_{\max} = 4$, $K = 2$, arrivals 0 or 1 with equal probability, $c_f = 3$, $c_h = 1$, and $\gamma = 0.8$, a separate loop enumerates all $2^5 = 32$ stationary deterministic policies. Each is evaluated by linear algebra using the reference transition builder. Taking componentwise minima is followed by checking that a single enumerated policy attains those minima at every state; this avoids assuming that a collection of statewise minima is attainable. The unique globally optimal policy is $(0, 0, 1, 1, 1)$ and its value is $(4.4, 6.6, 7.4, 9.6, 11.8)$. VI's sup error is 3.504×10^{-10} and PI matches the enumerated values to the reported precision. Both returned policies are also individually checked against the enumerated globally optimal policies.

1.5 Experiments E–G: demand, costs, and discounting

E: deterministic arrivals. Table 4 reports separately solved deterministic scenarios. Larger arrivals increase costs, but thresholds need not increase monotonically: they are 11, 13, and 11 for arrivals 1, 3, and 5. Fixed dispatch cost, capacity, and the integer sequence of reachable states interact. For example, with arrival 5, waiting at state 10 reaches a full load at 15; with arrival 3, the cycle from a full dispatch is $15 \to 3 \to 6 \to 9 \to 12 \to 15$. These observations concern the executed scenarios, not a general threshold theorem.

Arrivals	Threshold	$V^*(0)$	$V^*(200)$
1	11	311.50	3368.22
3	13	569.75	3819.21
5	11	794.30	4316.79

Table 4: Experiment E: deterministic passenger arrivals per period.

t	s_t	a_t	Cost	Arrivals	s_{t+1}	$V^*(s_t)$
0	15	1	100.0	3	3	669.753
1	3	0	6.0	3	6	599.740
2	6	0	12.0	3	9	624.989
3	9	0	18.0	3	12	645.252
4	12	0	24.0	3	15	660.265
5	15	1	100.0	3	3	669.753
6	3	0	6.0	3	6	599.740
7	6	0	12.0	3	9	624.989
8	9	0	18.0	3	12	645.252
9	12	0	24.0	3	15	660.265

Table 5: Ten-step optimal trajectory with deterministic arrivals 3 and initial state 15. Costs are assessed before the three new arrivals.

Every row obeys $V^*(s_t) = c(s_t, a_t) + 0.95V^*(s_{t+1})$ up to solver error. At state 15 the cost is 100 and all waiting passengers depart; at state 3 the controller pays holding cost 6 and waits. The cycle makes the benefit

of batching passengers visible without stochastic noise.

F: cost sensitivity. Free dispatch produces threshold 1; at zero passengers the free empty dispatch ties with wait. Its value at state zero is analytically zero: subsequent arrivals never exceed capacity and can be cleared without cost. PI's value differs from zero only by roundoff. With zero holding cost, never dispatch has value zero at every state and all other costs are nonnegative, proving optimality analytically. With $c_f = 1000$, never dispatch is optimal: a single dispatch removes at most 15 passengers and can save at most $15c_h/(1-\gamma) = 600$ in discounted holding costs. This follows by coupling the same arrivals and later actions with and without that dispatch; the population difference never exceeds 15. Charging 1000 cannot be justified. Increasing c_h to 10 instead lowers the threshold to 7.

Scenario	c_f	c_h	Threshold	$V^*(0)$	$V^*(200)$
Baseline	100	2	13	576.67	3821.59
Free dispatch	0	2	1	0.00	2417.94
Zero holding	100	0	Never	0.00	0.00
High dispatch	1000	2	Never	2203.91	8000.00
High holding	100	10	7	1123.93	13744.14

Table 6: Experiment F. "Never" means no state chooses dispatch.

G: discount-factor sensitivity. Larger γ increases nonnegative discounted costs and slows VI's contraction (Table 7). At $\gamma = 0.5$, dispatch can save at most $15(2)/(1-0.5) = 60 < 100$, so waiting is optimal everywhere. For $\gamma = 0.8, 0.95, 0.99$, detected thresholds are 14, 13, and 13. The larger future holding-cost weight makes earlier dispatch worthwhile in these scenarios, while VI effort rises from 43 to 2641 backups.

γ	VI backups	Threshold	$V^*(0)$	$V^*(200)$
0.50	43	Never	12.00	800.00
0.80	122	14	96.71	1875.86
0.95	519	13	576.67	3821.59
0.99	2641	13	3162.63	7137.91

Table 7: Experiment G: all discount scenarios also pass VI/PI policy agreement and residual checks. Values across rows represent different objectives.

1.6 Experiments H–I: simulation and action gaps

H: Monte Carlo policy validation. For each of four policies and initial states $0, 15, 50, 100, 200$, we simulated 20,000 independent episodes of 400 periods. The seed is 20261026 for the reported run. The policies share arrival draws to reduce noise in comparisons; samples across episodes within a policy are independent. Each policy is also evaluated exactly by a linear solve. The simulator directly applies the primitive event ordering instead of sampling the transition matrix.

The discounted simulated return includes terminal continuation $\gamma^H V_\pi(s_H)$, making it unbiased apart from numerical error. Raw truncated means and omitted-tail bounds are retained; the largest bound is 1.155×10^{-5} for the reported settings. The correction is far smaller than the stochastic standard errors. Consequently this is a simulation check of the transition/cost implementation, supplemented by the independent residual and exhaustive-policy certificates, rather than an independent proof of optimality.

Policy	s_0	Exact	MC mean	95% CI
optimal	0	576.67	576.59	[576.15, 577.02]
optimal	15	676.67	676.59	[676.15, 677.02]
optimal	50	1001.53	1001.46	[1001.03, 1001.89]
optimal	100	1709.04	1708.95	[1708.35, 1709.56]
optimal	200	3821.59	3821.50	[3820.46, 3822.54]
never_dispatch	0	2203.91	2203.97	[2201.73, 2206.21]
always_dispatch	0	2000.00	2000.00	[2000.00, 2000.00]
threshold_15	0	597.36	597.35	[596.89, 597.81]

Table 8: Selected simulation results; all 20 comparisons, standard deviations, errors, and tail bounds are in `monte_carlo.csv`. Intervals use $\bar{G} \pm 1.96\,\widehat{\sigma}/\sqrt{N}$.

All 17 of the 17 nondegenerate individual 95% intervals contain the exact value in this run. The 3 deterministic cases have zero sampling variance: never dispatch from 200 costs $400/(1-0.95) = 8000$ and always dispatch from 0 or 15 costs $100/(1-0.95) = 2000$. Their zero-width intervals agree with linear evaluation up to floating-point roundoff, explicitly recorded rather than counted as sampling failures. A Bonferroni normal-approximation check across all 20 cases uses $z = 3.023$ and all pass. These are statistical consistency checks; nominal 95% intervals need not all cover in another run. Exact evaluation also verifies that the computed optimal policy costs no more than every benchmark at *all 201 states*, not just the five simulated states.

I: action-value gap. Define $\Delta(s) = Q(s,1) - Q(s,0)$. Positive gaps favor waiting; negative gaps favor dispatch. The gap is +3.778004 at 12 and −4.099663 at 13, confirming a single switch with no near-ties. It is exactly 100 at zero because empty dispatch changes only the fixed cost. Near the cap the gap turns upward (approximately −275.39 at 195 and −208.92 at 200), because truncated arrivals make waiting artificially cheaper. It remains negative through 200, so the baseline policy has no second switch. This cap effect also shows why global monotonicity of the gap should not be asserted.

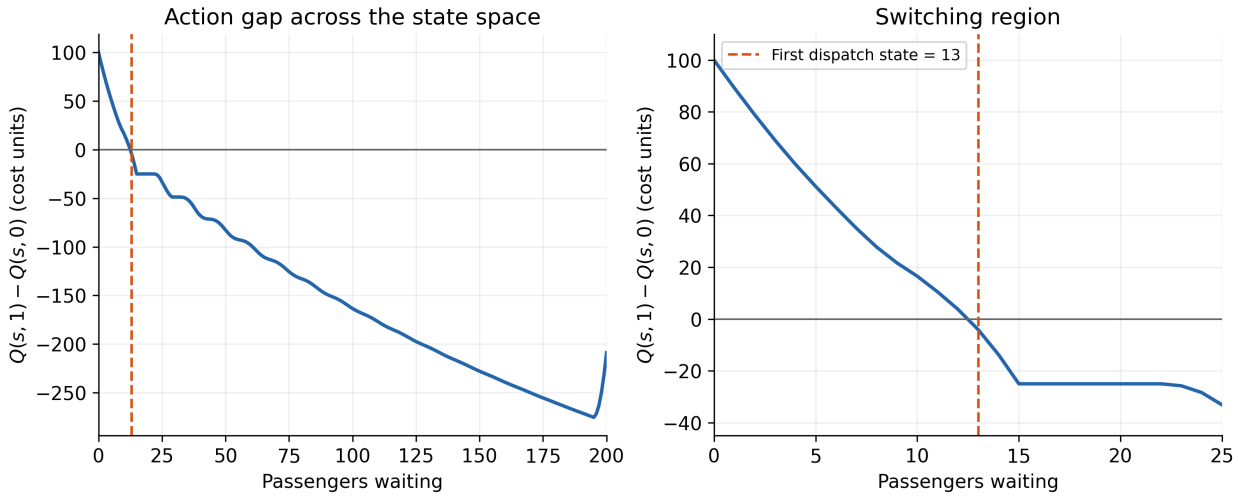


Figure 5: Experiment I: global gap and a magnified switching region. The boundary changes the gap's slope but does not reverse the baseline action.

1.7 Edge cases and numerical sensitivity

At $s = 0$, wait strictly dominates empty dispatch by 100. At states 1–12 the optimal action is wait; at 13–14 a partial dispatch is permitted and optimal. At 15 a dispatch leaves zero passengers, costs 100, and transitions to states 1–5. At 16 it leaves one passenger, costs 102, and transitions to 2–6. At 50 it leaves 35, costs 170, and transitions to 36–40. At 200 it leaves 185, costs 470, and transitions to 186–190. These explicitly checked remainders distinguish a one-shuttle action from clearing the entire queue.

At state 198, waiting reaches 199 with probability 0.2 and 200 with probability 0.8. Waiting at 200 stays at 200 with probability one. The production and independent reference matrices agree, including accumulated overflow mass. In the final finite-horizon decision, holding savings from one dispatch are at most 30, below the dispatch cost 100: every state waits and the last decision value equals $2s$. Earlier epochs account for future costs.

For ties, a separate immediate-cost model with $\gamma = 0$ and $c_f = 30$ has equal action values at every $s \geq 15$. Starting PI from all dispatch still returns all wait. Reducing c_f by 10^{-9} makes dispatch optimal at those 186 states, and the implementation selects it. With both costs zero, every action ties and PI returns wait everywhere from an all-dispatch initial policy. These tests exercise tie handling, not just the baseline's well-separated actions.

VI tolerance	Backups	Sup error against PI
1.000×10^{-6}	340	1.811×10^{-5}
1.000×10^{-8}	429	1.885×10^{-7}
1.000×10^{-10}	519	1.865×10^{-9}
1.000×10^{-12}	626	8.185×10^{-12}

Table 9: Convergence tolerance sensitivity. All four tolerances give the same baseline policy; residual-based bounds cover their value errors.

At very tight tolerances, roundoff can make the fresh residual slightly larger than the last successive difference; they are distinct diagnostics. Zero dispatch cost, zero holding cost, high dispatch cost, and alternative discount factors are analyzed above. Every important edge case has a recorded expected behavior, observation, implementation explanation, and check in `edge_cases.csv`.

1.8 Reproduction, interpretation, and limitations

From the repository root, install `requirements.txt` and execute `python scripts/run_value_policy.py`. This single command runs all algorithms and experiments A–I, saves the full finite solution and sensitivity arrays, produces 300-dpi figures, and regenerates the numerical LaTeX tables. The reported command uses the default seed, 20,000 episodes and 400 simulation periods. A quick run can reduce `-episodes`; it overwrites outputs and tables, so rerun the full default command before submission. All 18 of 18 numerical checks passed for the included artifacts. A failed check causes a nonzero exit.

The state, policy, convergence, trajectory, and simulation CSV files and the compact `summary.json` are in `results/value_policy_iteration/`. The summary records package versions, configuration, source hashes, runtimes, and checks. The exhaustive enumeration is independent of both solver loops; model arrays are also reconstructed independently, reducing the risk of agreement caused by shared errors. These checks establish strong numerical evidence for this specified finite MDP, not correctness of the shuttle model for real transportation operations.

Limitations include lost arrivals at the cap, known stationary demand, a fixed one-shuttle action space, and a zero terminal penalty. No general structural threshold theorem or empirical demand fit is claimed. Simulation uses a normal approximation for confidence intervals and a single reproducible seed; additional seeds would assess repeated-run coverage. Numerical solver error and statistical sampling error are reported separately. The source is modular so later policy-gradient components can join the same repository and report.

1.9 Validation plan for an additional three months

Month 1: demand and model calibration. Replace uniform arrivals with nonuniform demand, Poisson arrivals, and distributions estimated from passenger counts. Explicitly account for the tail when discretizing Poisson arrivals. Compare estimated distributions with held-out observations and bootstrap their uncertainty. Introduce time-of-day demand by augmenting the state with time, and assess whether stationarity materially changes decisions. These tests validate demand assumptions and sensitivity to estimation error rather than only the solver.

Month 2: boundary and operating-parameter robustness. Compare the present cap with larger state spaces, an overflow penalty, rejected passengers, and a queue that carries excess demand forward. Quantify changes in values and actions near and far from the boundary to measure truncation artifacts. Sweep dispatch and holding costs jointly and vary shuttle capacity; map policy switches and action gaps instead of presuming a threshold. Perturb arrival distributions and impose sustained surges to test distribution shift. These studies identify which conclusions survive plausible changes in costs, infrastructure, and demand.

Month 3: operational and out-of-sample validation. Evaluate frozen policies on held-out realistic demand traces, using multiple seeds, paired comparisons, repeated-run interval coverage, and both discounted cost and service measures such as waiting time and overflow. This tests generalization and exposes costs hidden by a single objective. Extend actions to multiple available shuttles and add availability or return time to the state where needed. Compare exact methods on tractable subsets with simulation and independent policy enumeration on smaller versions. This tests whether the state remains Markov and whether solver agreement persists when capacity constraints and action spaces become more realistic. These extensions are proposed validation work and have not been executed here.

2 Policy Gradient: Part 1

2.1 Estimator, implementation, and objective convention

The policy is a neural network producing two logits. Actions are sampled from a numerically stable categorical distribution, and Adam minimizes the manually implemented episodic REINFORCE loss

$$G_t = r_t + \gamma G_{t+1}, \quad t = T-1, \ldots, 0, \quad G_T = 0.$$

$$\mathcal{L}(\theta) = -\frac{1}{B} \sum_{i=1}^{B} \sum_{t} \log \pi_\theta(a_{it} \mid s_{it}) w_t G_{it}.$$

Time steps are summed; episode losses are averaged over the actual batch size, including the final partial batch. There is no baseline, critic, entropy bonus, return centering, reward normalization, or reward clipping. Gradient clipping is configurable but disabled in the executed reference experiments. Rewards in every required plot and evaluation table are raw, undiscounted episode totals.

The preserved CartPole results and earlier Pong pilots use $w_t = 1$, the relative-discount reference. The selected formal Pong run uses $w_t = \gamma^t$. The likelihood-ratio identity differentiates trajectory probabilities; past rewards can be removed from each action-score term because their conditional expected score is zero. This gives reward-to-go REINFORCE [2]. For theoretical precision, the strict initial-state discounted objective $J_\gamma = \mathbb{E}[\sum_t \gamma^t r_t]$ has likelihood-ratio gradient $\mathbb{E}[\sum_t \gamma^t \nabla \log \pi(a_t \mid s_t) G_t]$. These conventions are not identical. Only the absolute-weighting mode is the Monte Carlo gradient of this initial-state objective with the environment's finite episode boundary.

Sampling records the chosen action and its log probability without retaining the rollout graph. The same actions are recomputed under unchanged weights in chunks; gradients accumulate before one Adam step per batch. Networks contain neither dropout nor batch normalization. Each trajectory contributes to exactly one update. This gives the same summed estimator while limiting GPU graph memory; the numerical equivalence experiment below verifies that claim.

2.2 CartPole-v1

Completed: three training seeds, separate primary 500-episode evaluation. The model is $4 \to 128 \to 128 \to 2$ with ReLU hidden activations (17,410 parameters). The discount is 0.95, batches contain ten episodes, and Gymnasium's 500-step limit is preserved. Both failure termination and time-limit truncation end a rollout, with separate flags and zero continuation; there is no value bootstrap. The flags can overlap: the primary 500 evaluations contain 403 terminations, 99 truncations and two episodes with both flags, each counted only once.

The reference Adam rate is 0.01. Equal 300-episode pilots at seed 101 compared 0.01, 0.003, and 0.001. The largest last-100 training mean selected the learning rate before any final evaluation:

Learning rate	Final 100 training episodes: mean reward
0.01	49.45
0.003	82.96
0.001	43.91

Fresh training seeds 11, 22, and 33 each received 1500 episodes at rate 0.003. The budget is five pilot durations, permitting a test of sustained learning and regression. The primary seed (11) and its final-budget

checkpoint were predeclared. We did not select a seed or checkpoint by repeatedly evaluating the final set. The required stochastic evaluation uses exactly 500 episodes with reset seeds $1000000 + i$; its range is 278–500. An independent uniform-random reference uses reset seeds $3000000 + i$.

Policy	n	Mean	Sample SD	95% CI
Trained, seed 11	500	429.248	56.776	[424.27, 434.22]
Uniform random	500	22.700	13.379	[21.53, 23.87]

The mean improvement is 406.548 reward units, with normal-approximation 95% interval [401.435, 411.661]. This tests actual behavioral improvement, in addition to checking implementation. Intervals reflect episode sampling uncertainty for a fixed model, not uncertainty across all possible trained models.

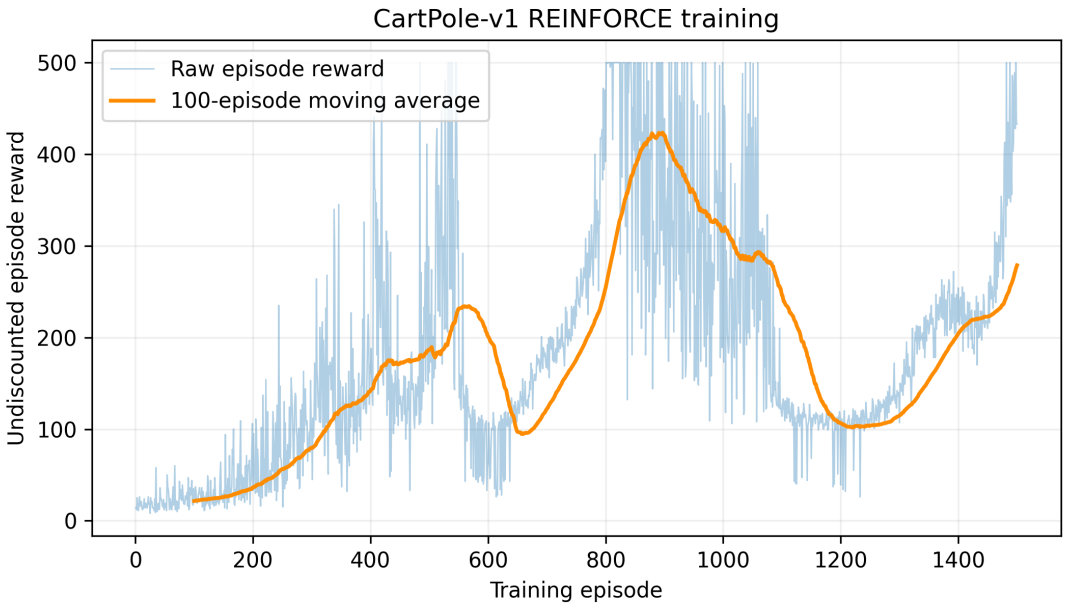


Figure 6: Primary CartPole training: raw rewards and the full-window 100-episode moving average, first plotted at episode 100. No evaluation reward enters this curve.

Figures 6 and 7 show the training trajectory and final reward distribution. Separate 100-episode diagnostic evaluations characterize independent training seeds:

Training seed	Diagnostic n	Mean	Sample SD
11	100	430.140	56.633
22	100	101.500	19.601
33	100	25.310	3.992

Figure 8 shows that learning-rate and training-seed effects remain substantial.

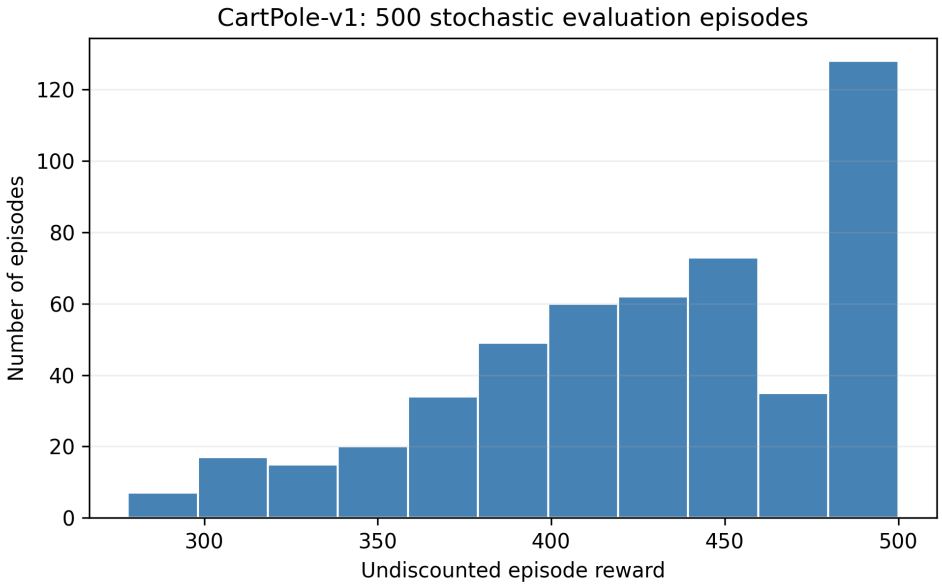


Figure 7: Exactly 500 independent-reset stochastic CartPole evaluations from the saved primary checkpoint; mean 429.248, sample SD 56.776.

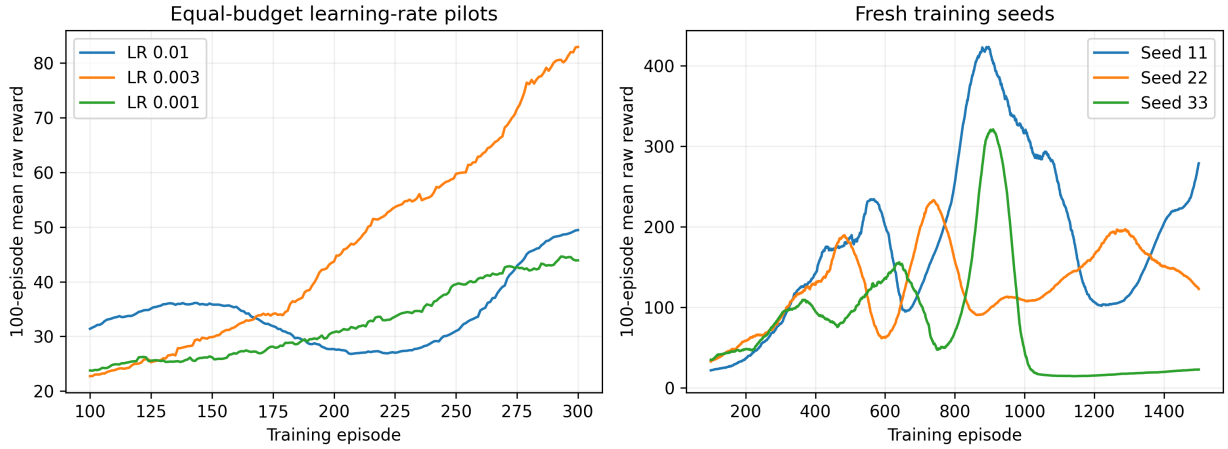


Figure 8: Learning-rate pilots and independent training seeds. The variability and regressions expose the instability of uncentered Monte Carlo updates. A strong primary model alone is not evidence of reliable convergence across seeds.

2.3 Pong-v5: environment, inputs, and pilots

The environment is exactly `ALE/Pong-v5`. Construction verifies the ordered six meanings NOOP, FIRE, RIGHT, LEFT, RIGHTFIRE, LEFTFIRE. Network indices 0 and 1 map only to actions 2 and 3. RGB frames have shape $210 \times 160 \times 3$. On a copy, rows 35–194 are cropped, alternate rows/columns and channel zero selected, colors 144/109 zeroed, and other nonzero values binarized into an 80×80 float array. Consecutive differences are an additional modeling assumption, explicitly compared with raw preprocessed frames; the first difference is zero.

The CNN uses convolutions $(1, 8, 8, 4)$, $(8, 16, 4, 2)$, $(16, 32, 3, 1)$, where tuples denote input/output channels, kernel and stride, followed by flattening 1152 features, a 64-unit hidden layer, and two logits. Hidden activations are ReLU; there are 81,146 parameters. The discount is 0.99, frameskip 4 and sticky-action probability 0.25. The ALE frame cap is 108,000 emulator frames. There is no life-based episode wrapper, reward clipping, or reward-to-go reset at points.

A real startup comparison completed games both with and without reset FIRE. Without FIRE, seed 73 produced 21 scoring events in 824 steps using only the two learned actions; therefore FIRE is disabled at reset and between points, equally in training and evaluation. This observation establishes progression, not learning.

Six affordable CPU pilots, ten full games each and batch size five, compare the assignment rate and two smaller alternatives under both input representations:

Input	LR	Episodes	Mean raw training reward
Difference	0.01	10	-20.40
Difference	0.003	10	-20.60
Difference	0.001	10	-20.40
Raw	0.01	10	-20.70
Raw	0.003	10	-20.60
Raw	0.001	10	-20.40

These contain only two policy updates each. A predeclared difference-input checkpoint at rate 0.003 is compared with reference policies using 30 diagnostic games per policy under identical environment settings:

Policy	Diagnostic n	Mean	Sample SD
Pilot policy	30	-20.500	0.938
Random	30	-20.400	0.814
Fixed	30	-21.000	0.000

These small pilots do not demonstrate meaningful Pong learning or establish an optimal representation. They verify complete games, action mapping, sparse rewards, gradient propagation, runtime, and checkpoint execution. The final Pong evaluation is a separate experiment and must reject truncated partial games.

Formal experiment and compute environment. The formal run used one RTX PRO 5000 Blackwell GPU on Deepdish5, isolated Python 3.12 and PyTorch 2.14.0+cu130, with Gymnasium 1.4.0 and ALE 0.10.2. The configuration was fixed at 1,000 games, Adam 0.01, gamma 0.99, absolute discount weighting, batch five and seed 11. Earlier short pilots used a different weighting convention and are retained as diagnostic evidence; their learning-rate selection is not presented as selection for this formal run. Training took approximately 1,166 seconds in this deployment and used 200 optimizer updates. Checkpoints were saved every 50 games.

A detached process then evaluated the fixed final checkpoint for 500 complete stochastic games. Only one GPU was visible, and training and evaluation were sequential. Configuration, source hashes, dependency versions, raw rewards and checkpoint identity were retained. Throughput is deployment dependent and is not an algorithm-quality result. The final-100 training mean was −20.65; the following independent evaluation establishes no meaningful Pong learning.

Completed 500 stochastic games: mean -20.608, sample SD 0.654, range -21—17. Training seed 11, evaluation seed 1000000.Figures 9 and 10 show near-minimum rewards throughout training and the final evaluation.

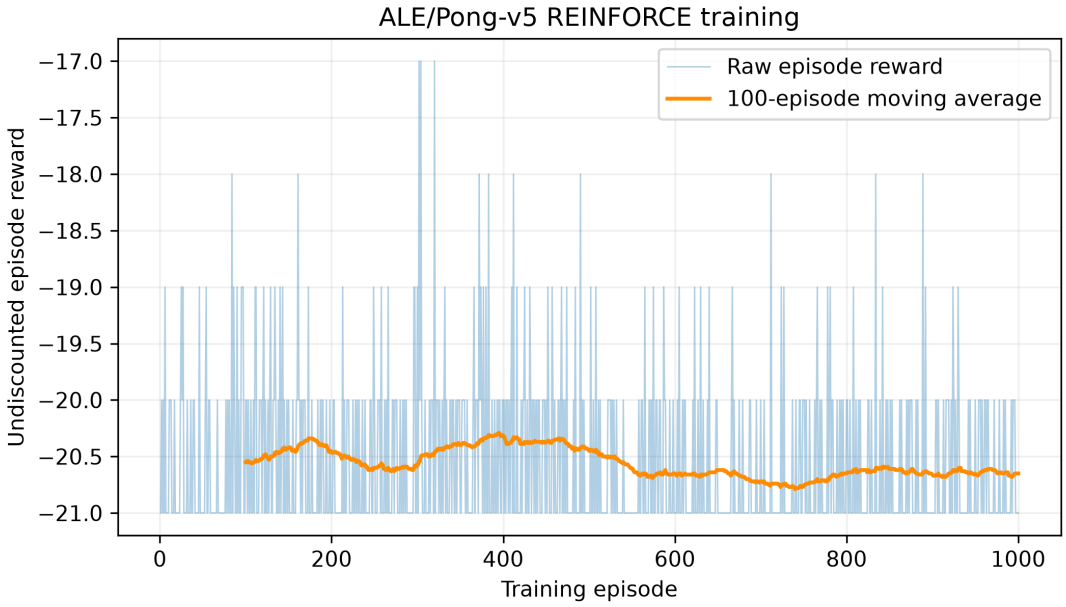


Figure 9: Recorded formal Pong training history: raw episode rewards and 100-episode moving average.

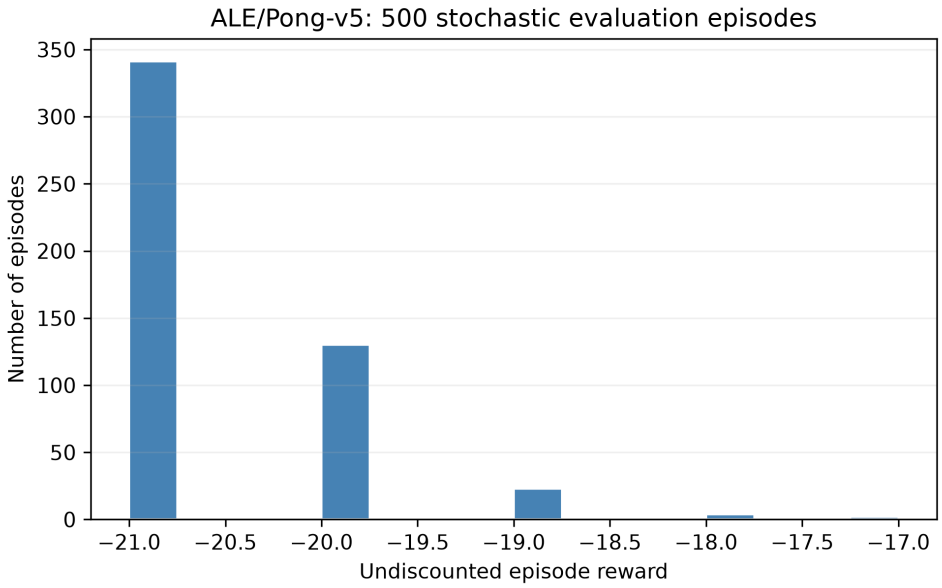


Figure 10: Independent 500-game stochastic Pong evaluation from the selected saved checkpoint.

2.4 Quantitative correctness and edge cases

Core validation passes 14/14 checks. In the one-step two-action problem, probabilities $(0.3, 0.7)$ and rewards $(1, -0.4)$ give loss gradient $(-0.294, 0.294)$. Exact action enumeration/autograd differs by 0; 100,000 Monte Carlo samples differ by 0.00116 at most. Independent finite differences agree within 10^{-8}. Handcrafted returns include $r = (1, 2, 3)$, $\gamma = 0.5$, giving $(2.75, 3.5, 3)$; gamma zero/one/0.999, a single step, and sparse signed rewards also agree to roundoff. A deterministic-reward sampled bandit increases the better action probability from 0.5 to 0.98562, validating ascent direction. Chunked gradients agree with the full-graph reference to 2.38e-07. Seven additional absolute-objective checks pass. In an exactly enumerated two-step task with shared action-one probability 0.3, rewards $r_0 = a_0$, $r_1 = 2a_1$ and gamma 0.5, the strict loss derivative is -0.42, matching finite differences; the relative-weighting derivative is -0.63. Absolute-mode chunked gradients also match the full graph to approximately 2.38×10^{-7}.

The same-CPU checkpoint test restores fixed-observation probabilities, sampled next-episode actions, Adam state, RNG state, and the resulting parameter update exactly. It does not promise restoration halfway through an emulator game or bitwise equivalence across CUDA/CPU or library versions. Periodic checkpoints save configuration, episode count, committed histories and source/version metadata. Reset seeds are explicit functions of training seed and episode.

Extremely unlikely sampled actions remain numerically valid: a logit gap of 2000 has finite loss and gradients. Nonfinite observations, logits, losses, gradients and parameter updates raise errors. Zero frame differences remain valid inputs; invalid dimensions and incorrect Pong actions fail explicitly. Short episodes and a forced one-step truncation validate boundary handling. Entropy, action probabilities/frequencies, gradient norms, loss, episode duration, termination flags and CUDA memory are logged so collapse and instability are visible. First-ten-step traces include observation summaries, actual environment action, log probability, raw reward, reward-to-go and both ending flags; raw images are not dumped into logs. Reproduction commands and complete evidence are documented in `docs/REINFORCE_PROTOCOL.md` and `docs/REMOTE_COMPUTE.md`.

2.5 Limitations and an additional three months

CartPole's short dense-reward episodes are much easier than Pong's sparse, long, image-based trajectories. No-baseline REINFORCE remains high variance: training improvement need not persist, a low entropy policy may collapse, and three seeds do not establish broad robustness. CPU emulator steps and serial action sampling can dominate this small CNN; GPU VRAM or theoretical FLOPS alone do not predict throughput. A correct gradient calculation does not establish sufficient training. Unsuccessful Pong learning must be reported as such.

With three more months, we would (i) increase independent training seeds and report uncertainty across models, (ii) compare learning rates, network sizes and budgets under equal environment-frame budgets, (iii) ablate raw versus difference inputs and motion ambiguity, (iv) test physical-parameter shifts in CartPole and initial-state/sticky-action shifts in Pong, (v) investigate sparse credit assignment using exact short multi-step problems and both discount conventions, and (vi) test interruption recovery under real preemptions. These respectively validate reliability, optimization choices, representation, robustness, estimator behavior and operational reproducibility. The next section extends this implementation with a running constant baseline for Policy Gradient Part 2.

3 Policy Gradient: Part 2 — running constant baseline

3.1 Method and matched experimental design

We reuse Part 1's policies, environment construction, action sampling, frame differences, chunked log-probability recomputation, Adam optimizer, checkpoint format, independent evaluation and plotting. No value network is introduced. For a complete trajectory, define

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k, \qquad \widehat{g} = \sum_{t=0}^{T-1} w_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)(G_t - b).$$

The loss is minus this score-function estimator, averaged over the update's episodes, with no length normalization, reward scaling or entropy bonus. For CartPole the primary comparison retains $w_t = 1$, $\gamma = 0.95$, Adam 0.003, ten episodes per update, 1,500 episodes and the same two 128-unit ReLU layers. For Pong it retains $w_t = \gamma^t$, $\gamma = 0.99$, Adam 0.01, five games per update and 1,000 episodes. Pong keeps the 81,146-parameter CNN, action indices $0 \mapsto 2$ (RIGHT), $1 \mapsto 3$ (LEFT), frameskip four, sticky probability 0.25, frame differences, no FIRE and no point-based discount reset. Only the baseline mechanism changes in these primary comparisons. The two weighting conventions are not pooled into a claim about one common objective: the absolute convention estimates the initial-state discounted gradient; CartPole retains Part 1's relative-discount reference convention.

The baseline is the exponential mean of *previously completed discounted episode returns* G_0, with $\alpha = 0.05$. It starts at zero for the first update batch. After that batch's optimizer step, its first observed G_0 initializes b directly; subsequent returns, in episode order, apply $b \leftarrow 0.95b + 0.05G_0$. A single scalar is frozen throughout every update batch. No return from the current batch affects any of its gradient weights. The policy, optimizer, RNG states, baseline value/count/alpha and committed history are saved together at update boundaries. Changing alpha or switching between baseline and baseline-free training on resume is rejected.

3.2 Why a constant baseline preserves the expected estimator

Condition on training history and the current state. The frozen scalar is independent of the current sampled action, so

$$\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}[b\nabla_\theta \log \pi_\theta(a \mid s)] = b\sum_a \nabla_\theta \pi_\theta(a \mid s) = 0.$$

Multiplication by deterministic w_t preserves this identity. Thus subtracting this baseline preserves the expected estimator for each convention, assuming finite returns and the usual differentiability conditions. It does not make the relative convention identical to the initial-state discounted objective. Individual sampled gradients differ. Using the current trajectory's own mean would invalidate this simple action-independence argument and is not done. The EMA is practical, not variance optimal: G_0 can be poorly scaled for late reward-to-go values, especially in sparse Pong games.

3.3 CartPole training and independent evaluation

Completed: primary 1,500-episode training and independent 500-episode evaluation.Primary seed 11 and its fixed-budget final checkpoint are declared before evaluation, matching Part 1; seeds 22 and 33 are independent robustness runs. Final evaluation samples 500 stochastic episodes with reset seeds starting at 1,000,000, in a separate invocation and without baseline subtraction. CartPole's standard 500-step TimeLimit counts as a complete capped episode; termination and truncation counts are saved separately.

Environment	Method	n	Mean	Sample SD
CartPole	Part 1	500	429.248	56.776
CartPole	Part 2	500	497.282	19.689
Pong	Part 1	500	-20.608	0.654
Pong	Part 2	500	-20.850	0.465

Table 10: Independent final evaluations of the four predeclared primary policies. Every row contains exactly 500 episodes; sample SD uses denominator 499.

For these primary checkpoints, mean reward changes by +68.034 and sample SD by -37.087. Final-100 training means are 278.76 versus 477.04; their reward SDs are 86.88 versus 55.26. Mean absolute successive changes in MA100 are 1.013 versus 1.130. These describe this matched primary pair, not population-level improvement across training seeds.

Seed	Diagnostic n	Part 1 mean	Part 2 mean	Part 2 SD
11	100	430.140	498.220	13.057
22	100	101.500	324.760	32.520
33	100	25.310	152.380	9.800

The separate 100-episode seed diagnostics use seed 2,000,000 in both parts. Substantial dependence on training seed remains; these diagnostics do not replace the primary 500-episode evaluation. The primary training curve also regresses mid-run, and its average absolute successive MA100 change is larger, so the stronger final score does not establish uniformly more stable optimization.Figures 12 and 13 show the primary baseline run and its final reward distribution. Figure 14 compares the matched primary runs; the advantage at the final checkpoint coexists with mid-training regressions. Figure 11 displays the remaining variation across seeds.

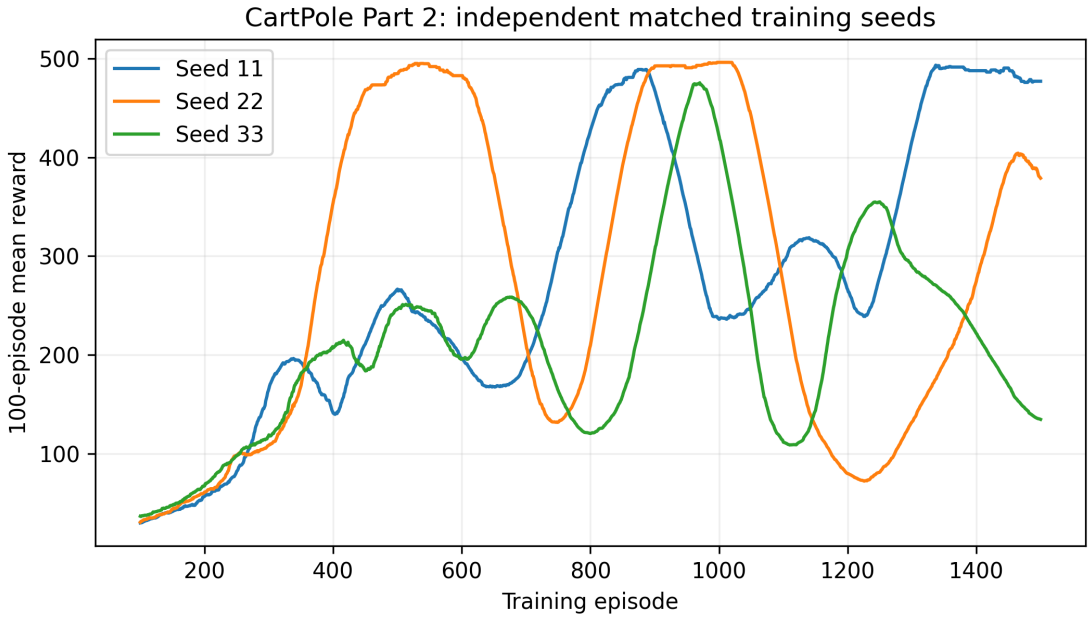


Figure 11: Independent Part 2 CartPole training seeds, each with the same 1,500-episode budget. Every curve is a full-window MA100; final performance depends strongly on the training seed.

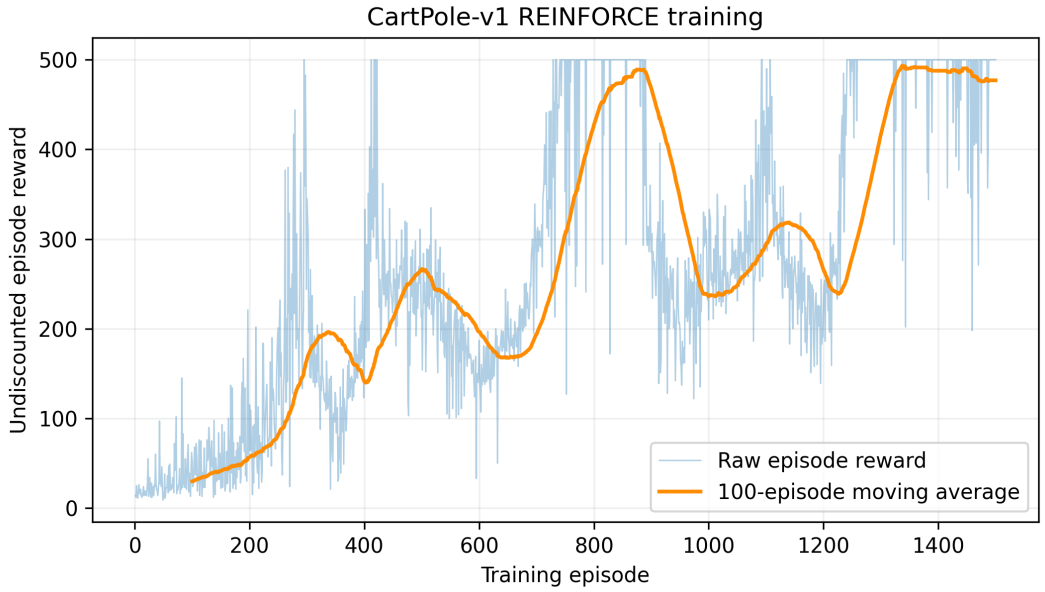


Figure 12: Part 2 CartPole primary run: raw rewards and full-window MA100.

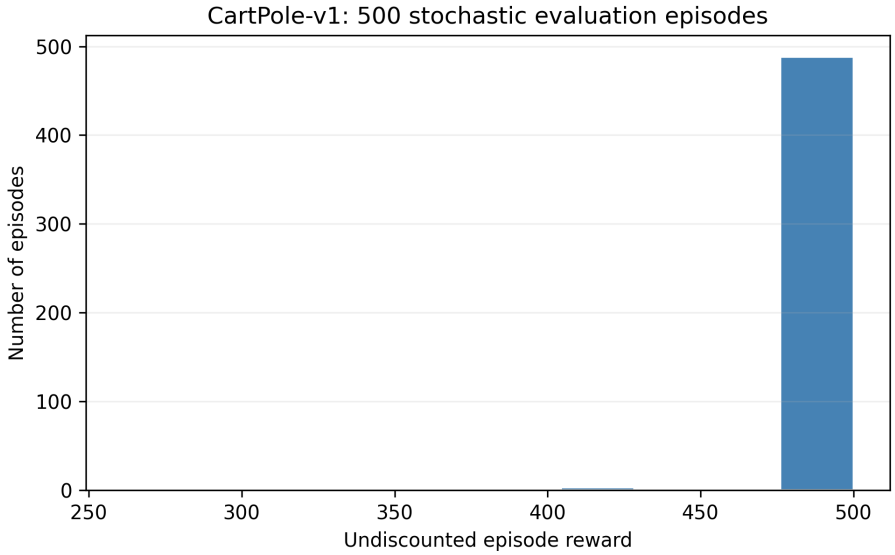


Figure 13: Exactly 500 independently evaluated CartPole episodes.



Figure 14: Matched primary CartPole runs and their separate 500-episode reward distributions.

3.4 Completed Pong training and independent evaluation

Part 1's verified completed run has final-100 training mean −20.65 and 500-game evaluation mean −20.608, sample SD 0.654. No saved Part 1 result is overwritten. A fresh release on Deepdish5 reuses the compatible Python 3.12 / PyTorch 2.14.0+cu130 user environment and physical GPU 1 (Blackwell), exposed alone as `cuda:0`. The GPU smoke completed four games, resumed with a nonzero restored baseline, and separately evaluated two diagnostic games. A shared account lock and an occupied-device check guard sequential GPU use. Formal execution used a detached process and durable logs, with checkpoints every 50 games. Training completed in approximately 1,143 seconds, followed by 500 complete stochastic evaluation games at seed 1,000,000, paired gradient diagnostics and an artifact archive. Retrieved outputs were checked against the recorded source manifest and evaluation checkpoint hash. All 1,000 training games and 500 evaluation games terminated without truncation. The final-100 training mean was −20.59.

Completed: 1,000 training games and exactly 500 complete final games. Mean -20.850, sample SD 0.465, range −21 to −18. No meaningful Pong learning is demonstrated.Figures 15 and 16 show that both training and evaluation remain concentrated near the losing score −21. Figure 17 contrasts the two formal runs. The baseline's final evaluation mean is lower by 0.242, despite measured gradient variance reduction; this is a negative performance result.

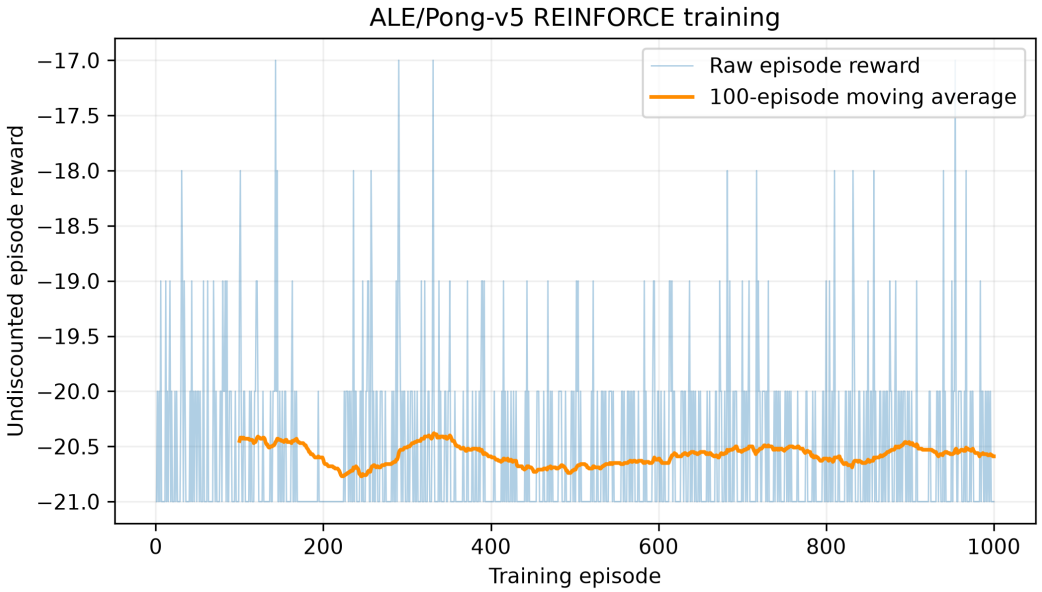


Figure 15: Actual completed Part 2 Pong training history and MA100.

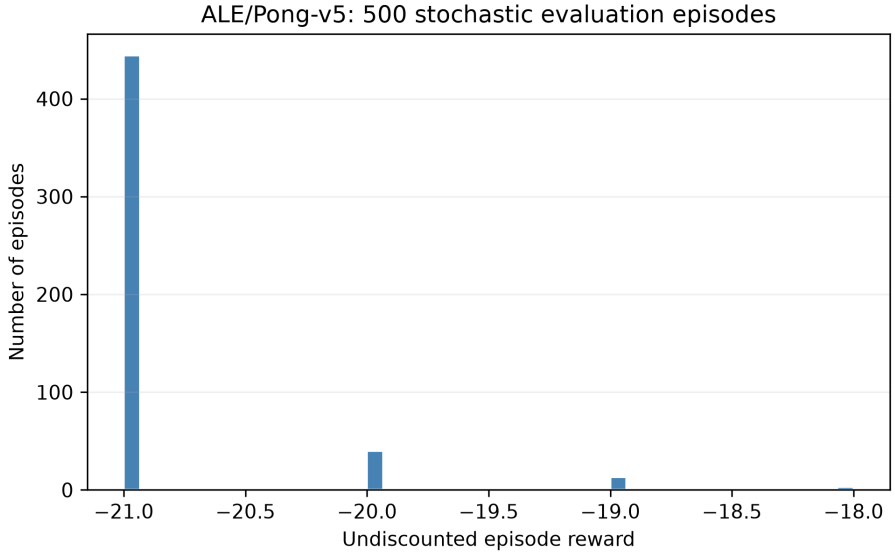


Figure 16: Actual independent 500-game Part 2 Pong evaluation.



Figure 17: Matched formal Pong runs, with absolute discount weighting in both parts.

3.5 Quantitative correctness and gradient variance

All 30 CPU checks pass. The largest exact expected-gradient discrepancy is 2.16×10^{-15}, and the maximum chunk/full-gradient error is 2.38×10^{-7}.Exact enumeration considers a two-decision, two-action problem with policy probabilities $(0.4, 0.6)$, reward tables $(0.2, 1)$ and $(1, 2)$, discounts 0, 0.95 and 1, both weighting conventions, and baselines $-10, 0, 2, 100$. It compares each expected estimator with the independent analytic gradient. Chunked baseline gradients also agree with full autograd, and a zero baseline is bitwise identical to Part 1's numerical path on the tested trajectory.

The synthetic variance experiment uses 200,000 paired independent one-step trajectories, policy $(0.25, 0.75)$ and rewards $(1, 3)$. Its frozen EMA is fitted on 2,000 disjoint prior trajectories. Both estimators use the same sampled actions, and covariance is measured across whole trajectory gradient vectors. The expected ascent gradient is $(-0.375, 0.375)$. The baseline is 2.297196; sample mean vectors are $(-0.374003, 0.374003)$ without and $(-0.375530, 0.375530)$ with the baseline. The paired mean-difference norm is 0.002160, with per-coordinate standard error 0.002226; finite-sample differences do not contradict equality in expectation.

Problem	Held-out n	No-baseline trace	EMA trace	Reduction
Synthetic	200000	0.845248	0.238744	71.75%
CartPole	32	1.92076e+06	44981.8	97.66%
Pong	32	1679.61	524.781	68.76%

Table 11: Paired whole-trajectory gradient covariance traces at a fixed policy. Synthetic trajectories use an independent prior EMA sample; the two game diagnostics use 32 held-out trajectories after 32 calibration trajectories. Each row compares estimators at one common policy, not across trained policies.

The game diagnostics freeze the final saved policy and its training EMA; they never update either using diagnostic returns. Of 64 independent complete trajectories (seed 4,000,000), 32 form a calibration sample and 32 a disjoint held-out sample. The latter estimates both paired gradient variances. Full per-parameter means/variances and paired mean differences are saved. A secondary constant $b^* = \operatorname{tr}\operatorname{Cov}(g, h)/\operatorname{tr}\operatorname{Cov}(h)$, where $h = \sum_t w_t \nabla \log \pi(a_t \mid s_t)$, is fitted only on calibration trajectories and assessed on held-out trajectories. It investigates baseline quality if the primary EMA fails; it is not used to alter formal training. For CartPole, the frozen EMA baseline is 20. Its paired mean-difference norm is 242.176, compared with estimated root trace of covariance of the paired mean 247.472. The independently calibrated secondary constant is 18.5156; its held-out relative variance reduction is 97.42%. This secondary constant was not used for training. For Pong, the frozen EMA baseline is -1.13056. Its paired mean-difference norm is 2.0721, compared with estimated root trace of covariance of the paired mean 3.50591. The independently calibrated secondary constant is -2.28534; its held-out relative variance reduction is 90.63%. This secondary constant was not used for training.Table 11 and Figures 18, 20 and 19 provide numerical and visual comparisons. The finite held-out samples support measured reductions at these fixed policies, not a universal or across-training guarantee.

3.6 Edge cases and reproducibility

Tests cover zero initialization, first-observation warm-up, negative returns, constant rewards, lengths 1, 2 and 10,000, discounts 0, 0.5, 0.95 and 1, nonfinite baseline inputs, missing/empty/mismatched/incomplete trajectories, chunk boundaries and baseline state restoration. Real training interruption at an update boundary reproduces model parameters, Adam state, RNG state, baseline and every numerical history column exactly on the same CPU/software. CartPole's normal TimeLimit is kept; shorter caps belong only to explicitly labeled correctness tests. Formal Pong has no artificial episode cap. Per-episode reset seeds are training seed

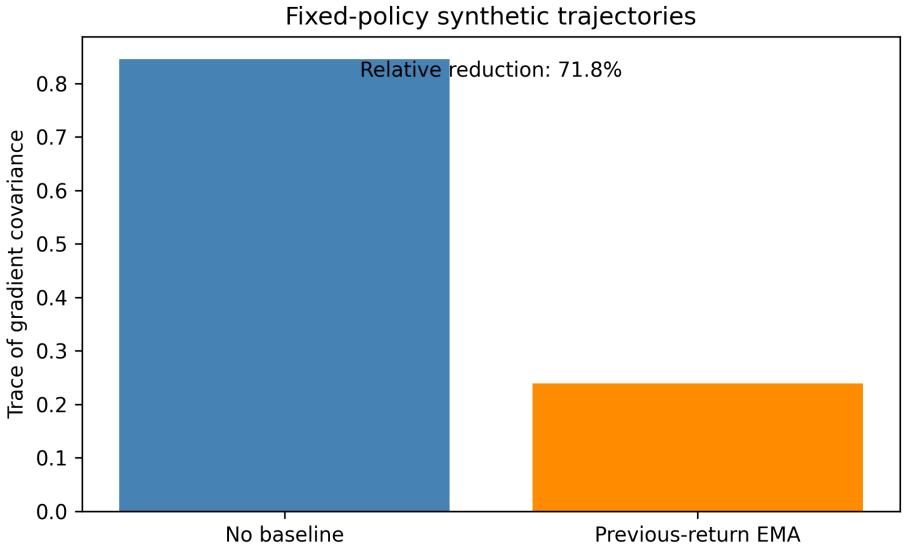


Figure 18: Synthetic paired-gradient variance using 200,000 independent one-step trajectories and a baseline fitted on disjoint prior trajectories. The exact expected gradient is invariant to the baseline.

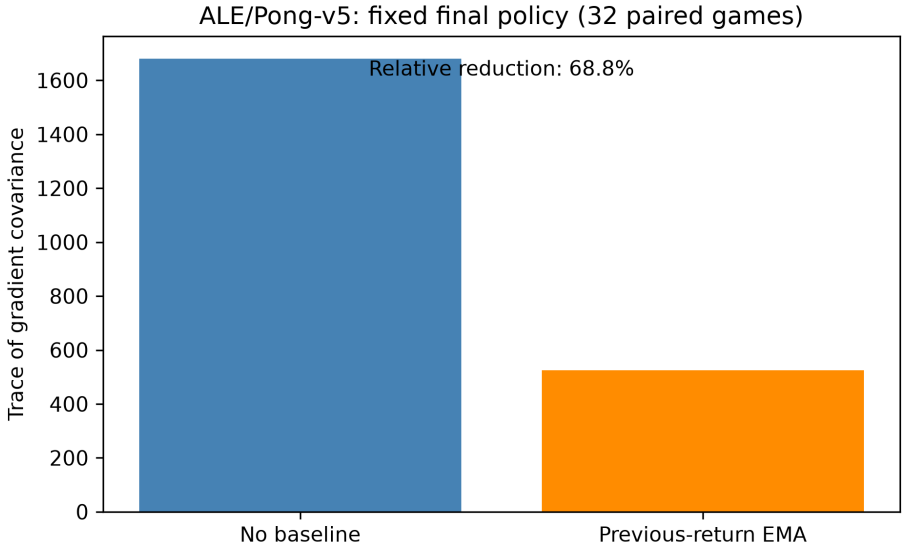


Figure 19: Pong gradient covariance trace on 32 held-out paired complete games at the fixed final Part 2 policy. The 68.76% reduction does not imply better final gameplay, as the independent evaluation demonstrates.

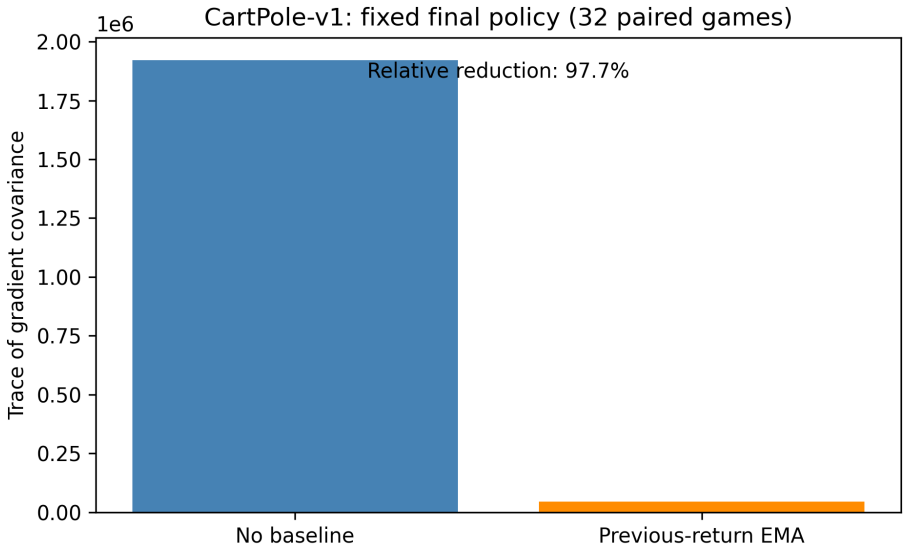


Figure 20: Paired whole-trajectory gradient variance at the fixed CartPole policy.

plus zero-based episode index. Configuration, package versions, model/environment metadata and source hashes are saved, along with baseline histories, gradient norms and log-probability recomputation errors. Independent evaluation records the checkpoint SHA256. Cross-device bitwise identity and mid-episode emulator restoration are not claimed. The readme provides training, evaluation, recovery and artifact commands.

3.7 Interpretation, limitations and three-month validation plan

Estimator variance is the covariance of gradient samples at one fixed policy. Episode reward variance describes a policy's rollout outcomes; training stability describes how performance changes during optimization; final performance is the independent evaluation mean. None of these four quantities proves the others. A smaller reward SD alone is not evidence of reduced gradient variance. A constant baseline may increase variance if poorly chosen, and an unbiased gradient estimator does not imply an unbiased Adam parameter update. The EMA tracks changing policies imperfectly. One primary seed per environment and 32 held-out diagnostic trajectories provide limited generalization evidence; matching seeds does not keep later trajectories identical once policies diverge. The baseline did not improve Pong reward within this budget. Absolute discount weighting can make long-delayed Pong credit extremely small at gamma 0.99; the baseline does not repair that estimator/budget limitation.

With three additional months: in month one, repeat matched baseline-free/EMA runs over at least ten seeds per environment, evaluate each fixed-budget policy for 500 episodes and report uncertainty across training seeds, not only across evaluation episodes. In month two, predeclare equal-frame-budget comparisons of EMA alpha and prior-return statistics against independently calibrated constant baselines; measure paired gradient covariance at initialization, mid-training and completion using substantially larger held-out samples. Test both discount conventions only with separately matched controls. In month three, test CartPole physical/initial-state shifts and Pong sticky-action/initial-state shifts, conduct real interruption/resume trials, and reproduce on another permitted GPU/software environment. Reserve held-out evaluations, report failed and negative runs, and quantify runtime and environment-frame costs.

4 Cross-experiment interpretation and extended validation

4.1 What the results establish

The shuttle problem admits strong numerical optimality certificates because its transition model is known and its state space is small. Independent Bellman residuals, exact policy evaluation and exhaustive enumeration on a reduced model support the solver results. They do not validate the realism of the population cap or arrival assumptions. In contrast, a correct policy gradient implementation provides no certificate that a finite training run has found a good neural policy. CartPole's strong primary score and the variation across seeds illustrate that distinction.

Diagnostic	Observed result	Inference and scope
Independent MDP residuals	VI 9.413×10^{-11}; PI 9.095×10^{-13}	Supports numerical Bellman optimality for the specified transition/cost model, not its real-world validity.
Reduced-MDP enumeration	All 32 stationary policies; unique optimum $(0, 0, 1, 1, 1)$	Checks solver outputs against an independent exhaustive reference on a tractable problem.
Part 1 exact gradient	Exact discrepancy zero; 100,000-sample discrepancy 0.00116	Supports the score-function implementation while exposing Monte Carlo sampling error.
Part 2 baseline invariance	Maximum exact discrepancy 2.165×10^{-15}	Subtracting a fixed action-independent baseline preserves the expected estimator in the enumerated tasks.
Chunking and continuation	Gradient discrepancy 2.384×10^{-7}; resumed CPU model/Adam/RNG/baseline/history identical	Supports memory-bounded backpropagation and update-boundary recovery in the tested software context.

Table 12: Selected quantitative validation evidence. Procedures and edge cases are developed in the corresponding component sections; passing these checks does not guarantee neural-policy convergence.

Table 12 separates implementation evidence from behavioral evidence. The final evaluation means in Table 10 measure the latter and must be interpreted with training-seed uncertainty.

For Pong, Part 2 changes the final evaluation mean by -0.242 reward units relative to Part 1, despite a 68.76% reduction in held-out gradient covariance trace at the fixed Part 2 policy. The variance comparison uses both estimators at one common policy, not the two independently trained policies. It therefore isolates an estimator property without claiming that the baseline causes better optimization. Both final policies remain near the minimum game score. The lower Part 2 reward SD also reflects its concentration near -21 and must not be interpreted as successful or more reliable play.

4.2 Production-path audit of Pong

A read-only audit compared the deployed execution paths with local sources and inspected actual configurations, histories and checkpoints. Small seeded CPU diagnostics used the production environment, preprocessing, sampling and loss functions without optimizer updates. No confirmed implementation defect was found in the tested paths. The following observations identify plausible limitations, while leaving their causal contributions unresolved.

Strong attenuation of late credit. Both formal Pong runs use the correct initial-state discounted estimator $\sum_t \gamma^t \nabla \log \pi(a_t \mid s_t) G_t$. The two discount factors inside $\gamma^t G_t = \sum_{k \geq t} \gamma^k r_k$ are not an accidental double-discount bug. However, at gamma 0.99 the outer factors at decisions 200, 500 and 1,000 are approximately 0.134, 0.00657 and 4.32×10^{-5}. Part 1 games averaged 1,002.421 decisions. In a seed-73 diagnostic using the final Part 1 checkpoint, the 885-step game had 82.6% of absolute coefficient mass and 95.9% of squared coefficient mass in its first 200 decisions. The loss-gradient norm was 0.4865 for decisions 0–199 and 0.00245 for decisions 500–884, using the production batch divisor five. Bin norms need not add because gradient vectors can cancel; squared coefficient mass is not itself gradient variance. These measurements demonstrate attenuation, not literal float32 underflow, and indicate a mismatch between strongly discounted training credit and undiscounted full-game evaluation.

Sprite preservation and observation ambiguity. The production crop and binarization retained ball pixels in all 570 frames where the ball was visible in the diagnostic's sampled raw playfield. Player and opponent paddle pixels also survived whenever present. The signed difference values remained $-1, 0, 1$, with no unsigned-subtraction error. Nevertheless, the player paddle disappeared from the difference region in all 96 observed unchanged-position cases. A single-channel feed-forward CNN has no memory with which to reconstruct an absent stationary paddle's absolute position. The first difference input is intentionally zero. Thus the preprocessing is implemented as specified, but the representation loses potentially useful state information.

Weak visual dependence of the trained policy. During the same diagnostic, action-0 probability ranged from 0.421926 to 0.423191, with SD 3.97×10^{-5}. Approximately 0.01% of third convolutional ReLU activations were positive on sampled inputs. On 64 identical scripted-input probes, that fraction changed from 37.2% at initialization to 9.4% at episode 50 and zero at episode 1,000. Some convolutional gradients remained nonzero on the full-game probe, so this is not evidence of a disconnected graph. It supports weak or largely inactive learned visual features; it does not prove that the learning rate or any particular component caused the condition. This checkpoint observation concerns Part 1 and is not automatically attributed to the Part 2 network.

Action, startup and alignment checks. Network indices 0/1 correctly selected environment actions 2/3 and produced opposite paddle movements. The no-FIRE diagnostic reached 21 scoring events and a terminated game, confirming startup and continued play. Captured raw frames reproduced every stored policy observation; chosen actions and rewards matched their corresponding environment steps. The loss sign reinforced positive-advantage sampled actions. Maximum production log-probability recomputation discrepancy was 2.38×10^{-7}. These checks reject several specific failure modes but cannot certify every possible execution.

Budget and causal limits. One thousand games in batches of five provide only 200 Adam updates to an 81,146-parameter policy, with sparse point rewards. The observed attenuation, representation ambiguity and weak image dependence make unsuccessful learning plausible within this budget. Their separate causal effects are not identified: the audit did not change algorithms or conduct new training comparisons.

4.3 A coordinated hypothetical three-month plan

The component-specific plans above detail model and environment tests. A coordinated extension would separate numerical correctness, model validity, estimator properties and policy performance as follows; none of these studies has been executed for this report.

Period	Proposed work	Evidence and decision criteria
Month 1	Expand independent training seeds; repeat MDP reference checks, simulation coverage and demand sensitivity.	Report uncertainty across models, paired policy comparisons and residual certificates separately from evaluation episode intervals.
Month 2	Predeclare matched, equal-frame-budget estimator and baseline comparisons; assess gradient covariance at multiple frozen checkpoints.	Use common trajectories within each estimator comparison, disjoint baseline calibration and larger held-out samples; record failures and compute cost.
Month 3	Test environment shifts, MDP caps and overflow assumptions, real interruption recovery and another supported software/device environment.	Freeze selection rules before held-out evaluation; measure generalization, reproducibility, service outcomes and final policy reward, without suppressing negative runs.

Table 13: Proposed additional validation, organized by the inference each study would support. Any change in discount convention requires its own matched baseline-free control.

Table 13 makes training seeds the unit of replication for algorithm comparisons. Longer budgets and hyperparameter sensitivity studies would be predeclared rather than selected using repeated final evaluations. The completed results support numerical solver agreement, successful primary CartPole learning and measured fixed-policy variance reduction; they do not support a claim of successful Pong learning or universal baseline superiority.

A Code Execution and Reproducibility

Run commands from the repository root using Python 3.10 or newer. Use a separate reproduction copy and fresh output directories: the Component 1 driver overwrites canonical outputs, and the Part 2 validator writes its synthetic figure. Code blocks below use Linux shell continuations (\); in PowerShell, join continued lines into one command. For tested package versions and remote-operation details, see `README.md` in the GitHub repository identified on the title page.

Environment setup. Create and activate an isolated environment, then install dependencies:

```
python -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements-pg.txt
```

On Windows, replace the activation line with `.\.venv\Scripts\Activate.ps1`. Component 1 alone needs only `requirements.txt`. Pong GPU runs require a compatible CUDA-enabled PyTorch build and exactly one allocated GPU exposed through `CUDA_VISIBLE_DEVICES`; the commands below use `cuda:0`.

Quick checks and correctness experiments. The first command checks saved results and report dependencies without sampling policies. The next two run bounded return/gradient, startup and checkpoint tests, including a tiny synthetic bandit and short resume tests; they do not repeat formal training.

```
python report/prepare_submission.py --check-only
python scripts/run_pg_validation.py --absolute --pong \
  --output tmp/pg1_checks
python scripts/validate_pg2.py --output tmp/pg2_checks
```

Inspect the validation JSONs in these output directories. Component 1's command below also executes all numerical checks and experiments A–I.

Component 1 and full policy-gradient training. The first command reproduces the saved MDP settings and figures. The four subsequent commands are computationally expensive training runs; configurations use seed 11 and preserve CartPole's relative and Pong's absolute weighting.

```
python scripts/run_value_policy.py --episodes 20000 \
  --simulation-horizon 400 --seed 20261026

python scripts/train_cartpole.py --learning-rate 0.003 \
  --episodes 1500 --output tmp/repro/pg1_cartpole
python scripts/train_pong.py --config configs/pong_part1_absolute.json \
  --episodes 1000 --device cuda:0 --checkpoint-interval 50 \
  --log-interval 10 --output tmp/repro/pg1_pong
python scripts/train_pg2.py --config configs/cartpole_part2.json \
  --baseline-alpha 0.05 --output tmp/repro/pg2_cartpole
python scripts/train_pg2.py --config configs/pong_part2.json \
  --device cuda:0 --baseline-alpha 0.05 --output tmp/repro/pg2_pong
```

PG runs save configuration, history and `latest.pt` in their output directory. For seed robustness, repeat with `-seed 22` and `-seed 33`; retain seed 11 as the predeclared primary run.

Independent 500-episode evaluation. After training, evaluate a saved checkpoint in a separate invocation:

```
python scripts/evaluate_pg.py \
  --checkpoint tmp/repro/pg2_cartpole/latest.pt \
  --output tmp/repro/pg2_cartpole_eval --episodes 500 --seed 1000000
```

For the other three runs, replace both occurrences of `pg2_cartpole` with `pg1_cartpole`, `pg1_pong` or `pg2_pong`; add `-device cuda:0` for Pong GPU evaluation. Final mode is stochastic by default and rejects truncated Pong games. Each new evaluation saves `rewards.csv`, mean/sample SD in `summary.json`, and a histogram.

Gradient-variance diagnostics and figure reproduction. This slower diagnostic samples 64 complete games at a fixed checkpoint (32 calibration, 32 held out), without training:

```
python scripts/diagnose_pg2.py \
  --checkpoint tmp/repro/pg2_pong/latest.pt --device cuda:0 \
  --episodes 64 --seed 4000000 --output tmp/pong_variance
python report/regenerate_figures.py --part 2 --environment pong \
  --output tmp/rebuilt_pg2_pong
```

The figure command reads the existing canonical saved CSVs, produces the raw reward/MA100 curve and 500-episode histogram, and never samples a policy. Use `-part 1` or `-part 2` and `-environment cartpole` or `-environment pong` to reproduce each pair in a fresh directory.

B Supporting dynamic-programming figures

The main text includes the three required value/policy plots, the horizon comparison and the action-value gap. The six additional saved figures below document convergence, agreement, sensitivity and simulation checks. They are supporting views of the executed experiments, not additional experiments.

Figure 21 shows declining VI successive differences and explains the stopping criterion's connection to the residual checks.

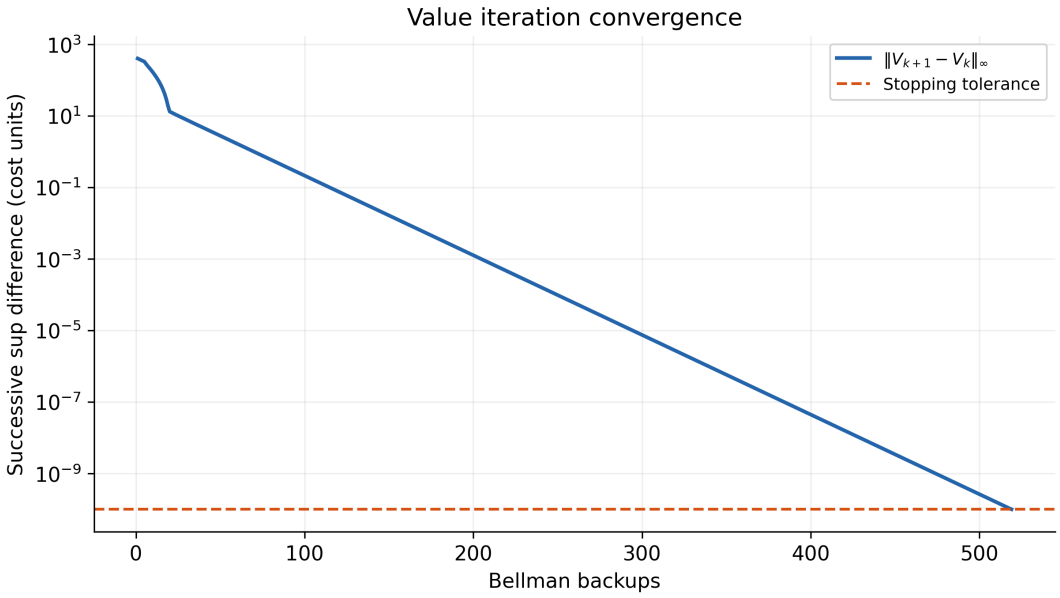


Figure 21: Saved VI convergence history for the baseline shuttle MDP. A small successive difference is complemented by an independently evaluated Bellman residual.

Figure 22 visualizes the VI/PI agreement quantified in the solver table; visual overlap alone would not certify correctness.

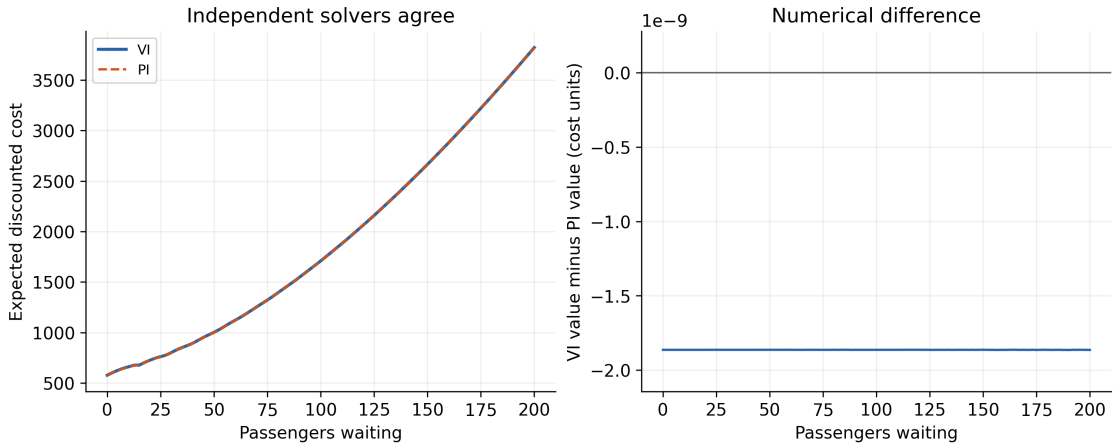


Figure 22: Value and policy agreement between independently structured VI and PI updates.

Figures 23 and 24 illustrate the cost and discount scenarios discussed in Experiments F and G. Each scenario

changes the objective and is solved separately; curves are not replicates of the baseline problem.

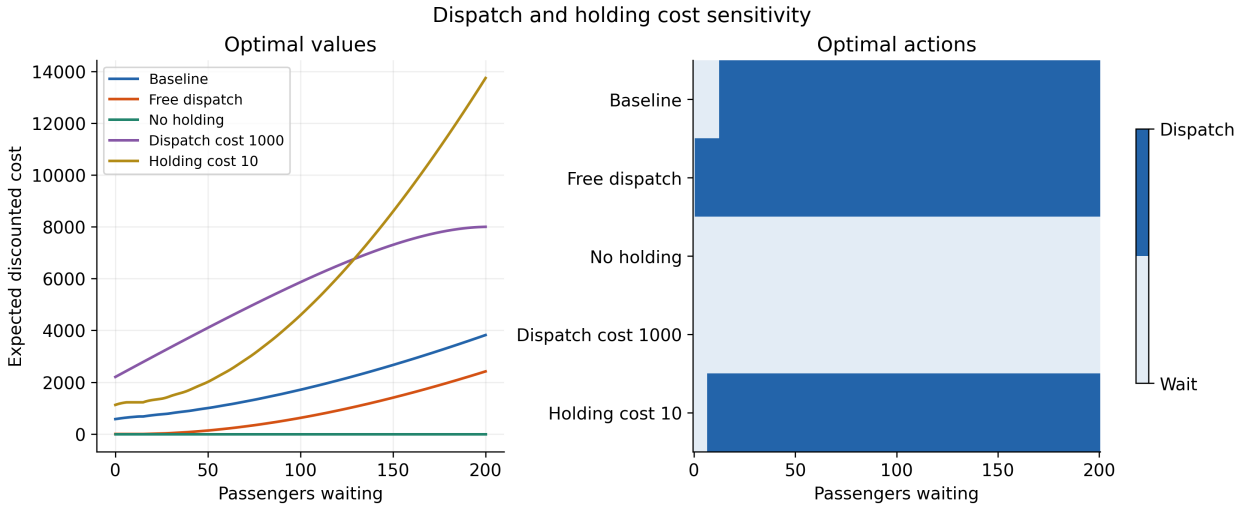


Figure 23: Executed dispatch/holding-cost sensitivity experiments, including free dispatch, zero holding cost and expensive dispatch.

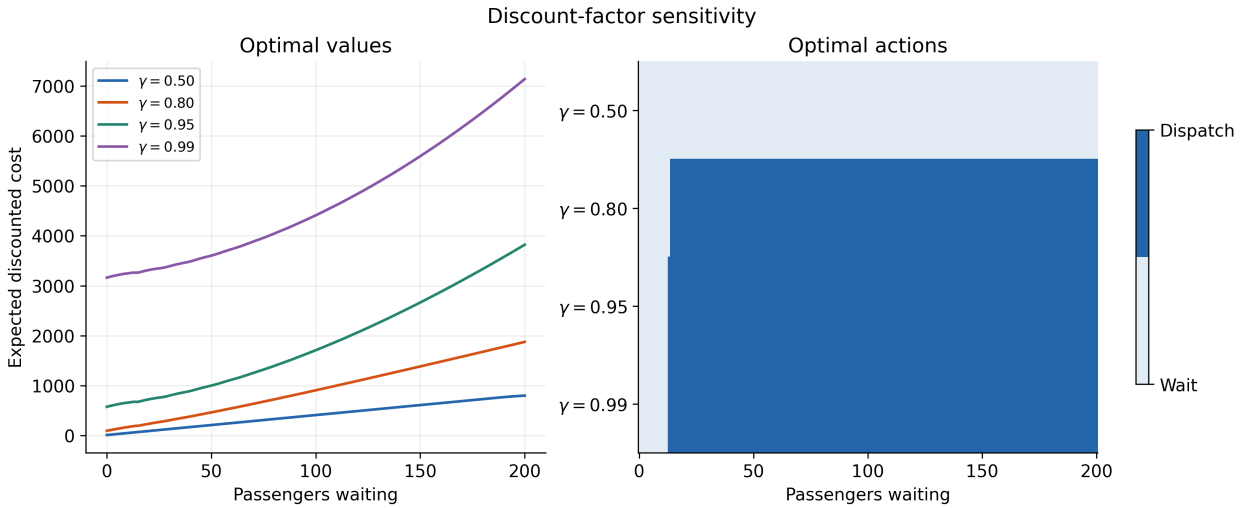


Figure 24: Executed discount-factor sensitivity: larger discounts increase the weight of future nonnegative costs and slow VI convergence.

Figure 25 supports the finding that detected thresholds need not vary monotonically with deterministic arrival size.

Figure 26 compares simulation with exact policy evaluation. Agreement tests the simulator; the residual and exhaustive-policy checks provide separate optimality evidence.

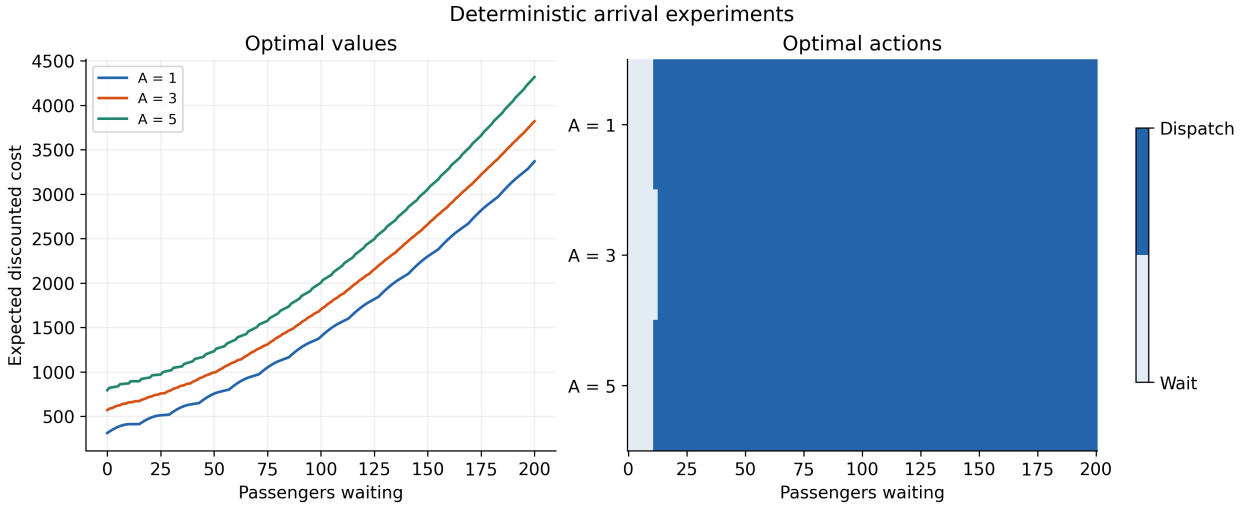


Figure 25: Optimal values and policies under deterministic arrivals of 1, 3 and 5 passengers, corresponding to Experiment E.

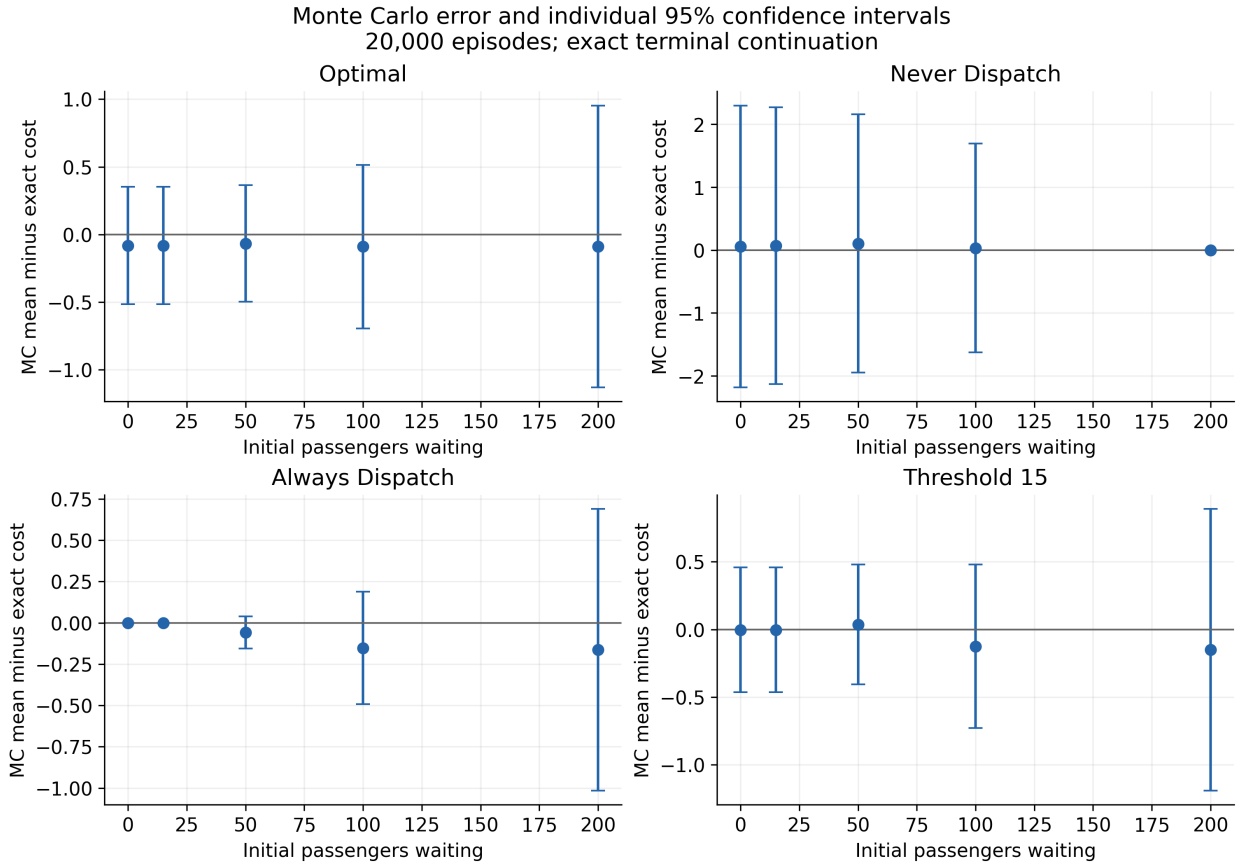


Figure 26: Monte Carlo policy-validation results against independently evaluated discounted costs, with the saved statistical uncertainty.

References

[1] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*, second edition. MIT Press, 2018. https://mitpress.mit.edu/9780262039246/reinforcement-learning/.

[2] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992. https://doi.org/10.1007/BF00992696.